

FTC StarterBot TeleOp Teacher Lesson Plan

Part 8 Applied Capstone Lesson

Middle and high school FTC students | 60 to 75 minutes | Teams of up to eight

This applied lesson brings the first seven TeleOp concepts together on a complete StarterBot. Students connect five configured devices to Java, predict drivetrain and intake behavior, rehearse the POWERING ON call-and-clear sequence, and test the program using evidence from telemetry. The robot remains secured with its wheels clear until every bench test passes.

Lesson outcome

By the end of the lesson, students can explain how one gamepad coordinates a skid-steer drivetrain, an intake roller, and two continuous-rotation feeder servos. They can also use calculated values and telemetry to justify a safe code or hardware correction.

Learning objectives

- Trace the iterative OpMode lifecycle through init, start, loop, and stop.
- Match five Java variables to exact Robot Controller configuration names.
- Calculate left and right motor power from forward and rotation inputs.
- Calculate reversible intake power by subtracting the left trigger from the right trigger.
- Explain why one mirrored feeder servo is reversed.
- Use the POWERING ON sequence and telemetry during a controlled robot test.

Required materials

- [] Completed StarterBot chassis with intake and two continuous-rotation feeder servos
- [] Control Hub, Driver Station, Gamepad 1, and charged 12-volt robot battery
- [] Stable robot safety blocks that raise both drive wheels and the intake clear of the table
- [] Laptop or Chromebook with access to OnBot Java or the team's Android Studio project
- [] Corrected StarterBotChassisTeleop.java supplied with this lesson
- [] Printed student workbook, focus guide, and POWERING ON safety poster
- [] Painter's tape for the test boundary and a clearly marked physical forward direction

Teacher preparation

- [] Inspect the battery, connectors, wiring, wheel retention, roller, and feeder servos.

- [] Secure the robot on stable blocks and verify that every moving part clears the surface.
- [] Confirm that the corrected program builds and appears in the Driver Station OpMode list.
- [] Verify all five configuration names before class; do not rely on memory.
- [] Mark a boundary that students do not cross while the robot is enabled.
- [] Open the video at <https://youtu.be/5OZR2QvJUuE> and test classroom audio.
- [] Prepare a clear floor-test area, but use it only after all wheels-up tests pass.

Exact hardware configuration

The quoted names in `hardwareMap.get` must match the Robot Controller configuration exactly, including underscores and capitalization.

JAVA VARIABLE	DEVICE TYPE	CONFIGURED NAME	PHYSICAL JOB
leftDrive	DcMotor	left_drive	Left drivetrain motor
rightDrive	DcMotor	right_drive	Right drivetrain motor
intake	DcMotorEx	intake	Intake roller motor
leftIntakeServo	CRServo	left_intake_servo	Left corner feeder
rightIntakeServo	CRServo	right_intake_servo	Right corner feeder

Student roles

Assign one role to each student. Rotate roles before the floor test so every student participates in both prediction and observation.

STUDENT	ROLE	RESPONSIBILITY
1	Driver	Uses small, deliberate gamepad inputs and returns controls to neutral.
2	Safety captain	Leads the POWERING ON sequence and watches the boundary.
3	Driver Station operator	Selects INIT, START, and STOP only when directed.
4	Code reader	Finds the matching Java statement for each test.
5	Hardware map inspector	Checks all five configuration names character by character.

STUDENT	ROLE	RESPONSIBILITY
6	Drivetrain observer	Records left and right wheel direction from outside the boundary.
7	Intake observer	Records roller and feeder motion from outside the boundary.
8	Telemetry recorder	Records gamepad inputs, calculated powers, and evidence.

Nonnegotiable safety boundary

No student approaches, holds, adjusts, or reaches into the robot while it is enabled. Wheel motion stopping is not permission to cross the boundary. The Driver Station operator presses STOP, the team confirms DISABLED, and the safety captain gives permission before anyone approaches.

POWERING ON sequence

1. Secure the robot with the wheels and intake clear of the surface.
2. Confirm the Driver Station shows DISABLED.
3. Look around the complete work area.
4. Call out loudly: POWERING ON!
5. Pause and give everyone time to step behind the boundary.
6. Visually confirm that hands, hair, clothing, tools, wires, and people are clear.
7. Select INIT or START only when the safety captain confirms the area is clear.

Video pause guide

Use the pause points as discussion stops rather than playing the video straight through. Times follow the published two-minute cut and may vary by about one second in the YouTube player.

PAUSE	TEACHER QUESTION	EXPECTED IDEA
0:09	What are the three robot jobs controlled by one gamepad?	Drive the chassis, run the intake roller, and move both corner feeders.
0:19	Which OpMode methods run once, and which method repeats?	init and stop run once; loop repeats after START.
0:31	Why must each quoted hardware name match exactly?	hardwareMap uses the quoted string to locate the configured device.
0:42	Why is one drive motor reversed, and what does BRAKE actually do?	Mirrored motors need opposite code directions to agree on forward. BRAKE reduces coasting; it is not a

PAUSE	TEACHER QUESTION	EXPECTED IDEA
		mechanical lock.
0:53	What must happen after the POWERING ON call and before START?	Everyone steps away, the operator pauses, and the safety lead visually confirms the entire area is clear.
1:14	Which controls provide forward and rotation?	The left stick Y controls forward and backward; the right stick X controls rotation.
1:26	Why does one drive equation add rotation while the other subtracts it?	Different left and right powers create the speed difference needed to turn.
1:39	What happens when both intake triggers are pressed equally?	Their values subtract to zero, so the intake command is zero.
1:48	How does one intakePower value coordinate three devices?	The same value is sent to the roller and both feeder servos; one mirrored servo is reversed.
1:56	How can telemetry help separate a code problem from a hardware problem?	It shows calculated commands and inputs so students can compare expected values with physical behavior.

Lesson flow

STEP	ACTIVITY	TIME	TEACHER ACTION	STUDENT ACTION
1	Launch and predict	5 min	Show the StarterBot and ask students to name every device one gamepad must control. Do not power the robot.	Students identify two drive motors, one intake motor, and two feeder servos.
2	Watch with pauses	12 min	Play the video and use the pause guide. Record student predictions before revealing answers.	Students annotate the focus guide and explain each calculation aloud.
3	Audit the hardware map	8 min	Display the code beside the Robot Configuration. Read each line from the quoted name back to the Java variable.	Students compare every character and locate the matching physical device.
4	Calculate arcade drive	8 min	Assign the drive cases. Require raw left and	Students compute motor powers and

STEP	ACTIVITY	TIME	TEACHER ACTION	STUDENT ACTION
			right calculations before discussing normalization.	predict straight, turning, or curved motion.
5	Calculate intake control	6 min	Model the triggers as a tug-of-war using right minus left.	Students calculate intakePower and predict inward, outward, or stopped motion.
6	Run the safety sequence	5 min	Position the robot on blocks, establish the boundary, and let the safety captain lead every step.	Students step clear, wait through the pause, and confirm the area visually.
7	Perform wheels-up tests	12 min	Authorize one input at a time. Stop after each category and compare observation with telemetry.	Students record direction, command values, and evidence without crossing the boundary.
8	Floor test and debrief	10 min	Move to the floor only after the bench test passes. Rotate roles and run low-speed drive and intake checks.	Students complete the exit response and identify one evidence-based improvement.

Hands on walkthrough

Hardware map audit

Keep the robot disabled. The code reader reads one hardwareMap line. The inspector finds the exact configured name. The observer points to the physical device. The recorder marks PASS only when all three agree.

Teacher question: What evidence proves that leftDrive reaches the left drivetrain motor?

Arcade drive prediction

Give the driver a forward and rotation card. Students calculate leftPower and rightPower. The drivetrain observer predicts wheel directions before the operator enables the robot.

Teacher question: If forward is 0.60 and rotate is 0.20, why does the robot curve rather than rotate in place?

Intake prediction

Assign right- and left-trigger values. Students subtract left from right and predict all three intake devices. The intake observer checks whether both feeders pull inward together.

Teacher question: Why does reversing one feeder servo make the physical motions agree?

Controlled test

Run POWERING ON. Use one small input, return to neutral, and press STOP. Confirm DISABLED before discussing any physical adjustment.

Teacher question: Which evidence came from code, telemetry, and physical observation?

Troubleshooting

Change only one hypothesis at a time. Repeat the prediction, safety sequence, observation, and evidence record after each code or configuration change.

Teacher question: What is the smallest safe test that can confirm or reject the hypothesis?

Wheels up test order

- [] INIT completes without a hardware-map error.
- [] Telemetry reports Status Initialized.
- [] Left drive command turns only the expected side in the expected direction.
- [] Right drive command turns only the expected side in the expected direction.
- [] Forward command makes both sides agree on physical forward.
- [] Rotation command makes the sides oppose correctly.
- [] Right trigger moves the roller and both feeders inward.
- [] Left trigger reverses the roller and both feeders.
- [] Equal trigger values command zero intake power.
- [] STOP is pressed and DISABLED is visually confirmed before approach.

Technical notes

Iterative OpMode. The FTC runtime calls init once, loop repeatedly after START, and stop once at the end. Empty init_loop and start methods remain valid lifecycle extension points.

Motor direction. Mirrored drivetrain motors often need opposite software directions. Direction must be verified on the actual robot because mounting and gearing can change the required setting.

BRAKE behavior. BRAKE reduces coasting when zero power is commanded. It must not be described as a mechanical parking brake or permission to approach an enabled robot.

Continuous rotation servos. CRServo power controls direction and speed, not an absolute position. Reversing the right feeder makes mirrored mechanisms pull inward together.

Trigger subtraction. right_trigger minus left_trigger creates a proportional command from -1.0 to +1.0. Equal trigger values cancel to zero.

Telemetry. Telemetry reports commands and inputs, not proof that the physical device moved correctly. Diagnosis requires comparing code, telemetry, and observation.

Normalize arcade drive output

The raw equations can produce values outside the motor-power range. Normalize both values by the same maximum magnitude to preserve their ratio.

```
leftPower = forward + rotate;
rightPower = forward - rotate;

double maxMagnitude = Math.max(
    1.0,
    Math.max(Math.abs(leftPower), Math.abs(rightPower))
);

leftPower /= maxMagnitude;
rightPower /= maxMagnitude;
```

Correct trigger telemetry formatting

```
telemetry.addData(
    "Triggers",
    "left (%.2f), right (%.2f)",
    gamepad1.left_trigger,
    gamepad1.right_trigger
);
```

Assessment evidence

- Completed configuration-name audit with all five mappings correct
- Accurate arcade-drive and trigger calculations
- Observed compliance with the POWERING ON sequence
- Wheels-up test record comparing prediction, telemetry, and movement
- Exit response explaining how one gamepad controls five devices

Exit question

How does this program turn one gamepad into three coordinated robot jobs, and what evidence would you check first if one job behaves incorrectly?